En un enfoque tradicional el testing se empieza a ver casi al final, luego de requerimientos, implementación.
Como por ejemplo el acoplamiento incidental, los desarrolladores creaban las cosas creyendo que todo iba a funcionar, se debian generar todo un contexto para poder probar

Deuda técnica y miedo al cambio, a la hora de salir a producción siempre salían con algún error

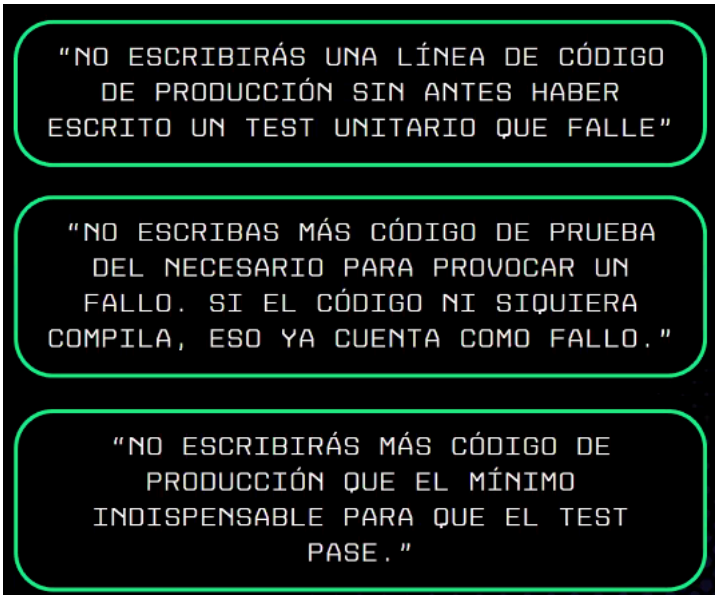
Costo asintótico del error, a medida que pasa el tiempo el costo se incrementa exponencialmente

luego nació TDD a causa de todo esto. Plantea una práctica en donde se cambia el paradigma
TDD NO ES UNA PRÁCTICA DE TESTING, es una práctica de desarrollo que utiliza testing para desarrollar.

TDD: Es una actividad de especificación y diseño que permite guiar el desarrollo por microciclos de retroalimentación, por medio de pruebas definimos que tiene que hacer el software y luego pasamos al como

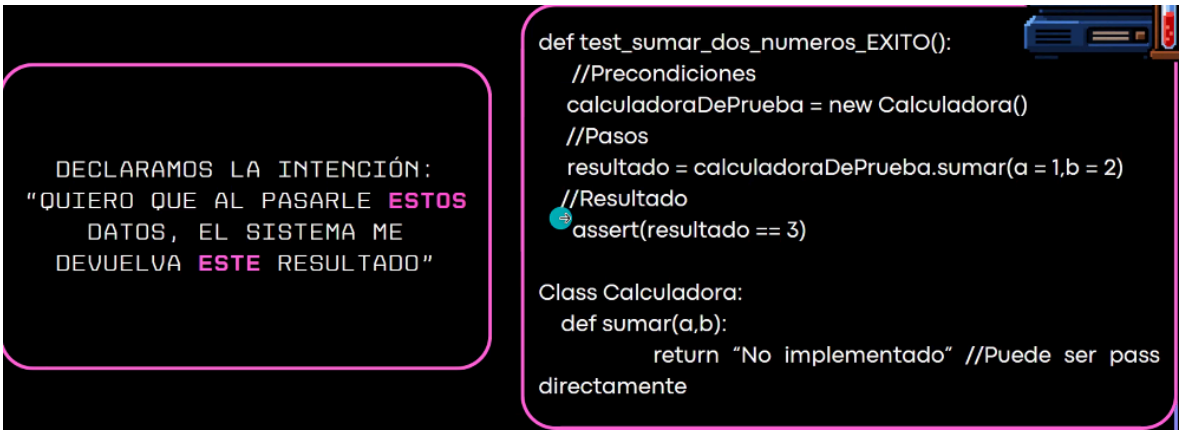
Primero defino las pruebas que despues voy a desarrollar,

Las 3 leyes de TDD



Las 3 etapas de TDD (Ciclo ciclico)

- RED: Aca escribimos una prueba que falle, es una declaracion de lo que queremos hacer. Declaramos las entradas y la SALIDA o resultado que esperamos con esa prueba. La idea



- GREEN: Ahora pasamos a un enfoque IMPERATIVO (el COMO), acá no nos enfocamos en calidad (Patrones o principios de diseño) y buscamos escribir el mínimo código para que la prueba pase.
  Hay patrones que nos ayudan a definir como pasar de una etapa a otra.
  Ej: “Fakeit” que se trata de harcodear

- REFACTOR: es una reestructuracion de buenas practicas, aca volvemos a aplicar patrones de diseño y principíos.
  NO MODIFICO EL COMPORTAMIENTO,

Luego comienza un nuevo ciclo, ahora en los test modifico los parametros de entrada, por ende los resultados, y en ese punto la funcion de test va a fallar, esto nos va a indicar que hay que generalizar lo que habiamos hecho (El harcodeado)

Patrones:

- Patrones red
- Patrones green
- Patrones de diseño de test
- Patrones que ayudan a las etapas

PATRONES DE RED: ayudan a definir que queremos hacer

- Starter test: Nos indica que tenemos que hacer el test lo mas simple posible para que el flujo comience “Elegir de toda la lista de alternativas la mas sencilla o trivial” “una prueba sencilla que nos permita comenzar a definir el comportamiento esperado. “

- One step test: Una vez que arrancamos hay que seguir con el pasito siguiente y no saltamos de un test facil a uno más complejo

- Explanation test: Nos propone que al debate del equipo lo planteemos como un test para hacer frente a la diferencia de criterio. Nos ayuda a resolver la problemática de entender algo

- Learning test: Cuando queremos conectarnos con un sistema externo/librería o código externo, hay que representarlo como un test (Similar al explanation pero cuando usamos herramientas externas)

- Another test: la idea es que cuando nos distraemos de lo que estamos haciendo y vemos que hay otro posible caso, lo anotemos para no olvidarnos

- Regresion test: Crear una prueba que reproduzca un error encontrado, para comprobar su corrección y evitar que vuelva a aparecer. Nos permite replicar el entorno en donde se produjo un error.

PATRONES GREEN

- Fake it: Vamos a harcodear el retorno para devolverme lo que la prueba define como resultado
- Triangulacion: Armar varios casos de falla, para definir que la falla esta en la logica que yo desarrolle.
  “Consiste en agregar nuevas pruebas con diferentes datos para obligarnos a generalizar una implementación que inicialmente era demasiado específica. “

- Obvious Implementation: Cuando la solución es evidente y sencilla, (o el desarrollador ya sabe exactamente cómo desarrollar) podemos implementarla directamente, sin necesidad de pasar por Fake It.

- One to many: Consiste en comenzar resolviendo un caso individual y luego extender la solución para que funcione con una colección de elementos.
-

PATRONES DE DISEÑO DE TEST:

- Test list: Me siento y creo todos los test que necesito enumerandolos en una lista

- Isolated test: Los test deben ser independientes de otros test

- Test First: Consiste en escribir primero la prueba y después desarrollar el código que permita superarla.

- Assert test: Al escribir una prueba, comenzamos definiendo la afirmación (assert) que indica qué resultado esperamos obtener. Luego preparamos lo necesario para verificarlo.

- Test Data: Utilizamos datos de prueba sencillos, fáciles de interpretar y seguir.

- Evident Data: Los datos utilizados deben permitir identificar claramente la relación entre las entradas y el resultado esperado. Que cualquier persona pueda comprender qué verifica la prueba y por qué se espera ese resultado.

Diferencia importante: **Test Data** se enfoca en utilizar datos fáciles de comprender, mientras que **Evident Data** busca que sea evidente por qué esos datos producen el resultado esperado.